



Universidade do Minho

Braga, Portugal

GUIÃO DE MANUTENÇÃO

LI4

EcoRide

G07

Engenharia Informática 2025/26

Equipa de Trabalho:

A106936 - Duarte Escairo Brandão Reis Silva

A106932 - Luís António Peixoto Soares

A106856 - Tiago Silva Figueiredo

A104704 - Inês Ferreira Ribeiro

27 Maio 2026

Índice

1. Introdução	1
1.1. Introdução	1
1.2. Visão Geral da Arquitetura	1
1.3. Pré-requisitos e Ambiente	2
1.3.1. Requisitos para execução em Docker (recomendado)	2
1.3.2. Requisitos para desenvolvimento local	3
1.3.3. Variáveis de ambiente	3
1.4. Arranque e Paragem do Sistema	4
1.4.1. Primeira execução	4
1.4.2. Execuções subsequentes	5
1.4.3. Paragem	5
1.5. Base de Dados	5
1.5.1. Inicialização automática	5
1.5.2. Estrutura do esquema	6
1.5.3. Acesso direto à base de dados	6
1.6. Cifragem de Dados Sensíveis	7
1.6.1. Campos cifrados	7
1.6.2. Algoritmo: <i>AES-256-GCM</i>	7
1.6.3. Gestão da chave de cifragem	9
1.7. Segurança e Autenticação	9
1.7.1. Middleware de autenticação	10
1.7.2. Cargos e permissões	11
1.7.3. Sessões	11
1.8. <i>API REST</i>	12
1.8.1. Estrutura de rotas	12
1.8.2. Tratamento de erros	12
1.8.3. Adicionar um novo <i>endpoint</i>	12
1.9. 8. <i>Frontend</i>	13
1.9.1. Configuração do <i>nginx</i>	13
1.9.2. Build local do <i>frontend</i>	13
1.9.3. Processo de <i>build</i> em Docker	14
1.10. Execução de Testes	14
1.10.1. Executar os testes	14
1.10.2. Padrão de <i>DAO</i> falso em memória	14
1.10.3. Adicionar testes a um subsistema existente	15
1.11. Adição de um Novo Subsistema	15
1.12. Monitorização e Logs	16
1.12.1. Visualizar logs em tempo real	16
1.12.2. Mensagens de arranque relevantes	16
1.12.3. Estado dos containers	16

1. Introdução

1.1. Introdução

O presente guia destina-se a técnicos e programadores responsáveis pela manutenção, operação e evolução do sistema **EcoRide** — uma aplicação de gestão de oficina de trotinetes elétricas. O objetivo deste documento é fornecer uma referência completa e estruturada para quem necessite de colocar o sistema em funcionamento, realizar alterações ao código, resolver problemas operacionais ou estender as funcionalidades existentes, sem necessidade de ler a totalidade do código-fonte.

O guia encontra-se organizado de forma progressiva: começa pela visão global da arquitetura, desce até aos detalhes de configuração e base de dados, aborda os mecanismos de segurança, explica a *API REST* e o *frontend*, descreve a estratégia de testes, e termina com procedimentos de extensão e resolução de problemas. Cada capítulo é autónomo mas a leitura sequencial é recomendada para quem se familiarize com o sistema pela primeira vez.

Os principais conceitos abordados ao longo do documento incluem: a **arquitetura em três camadas** ($CA \rightarrow LN \rightarrow CD$), a organização em **8 subsistemas** com Facades independentes, a **containerização com Docker**, a **cifragem de dados pessoais** com *AES-256-GCM*, o sistema de **autenticação por token de sessão**, a **especificação OpenAPI** e os **testes unitários automatizados** com *JUnit 5*.

1.2. Visão Geral da Arquitetura

O EcoRide segue uma arquitetura em três camadas horizontais, claramente separadas por pacotes Java e por responsabilidade:

- **CA — Camada de Apresentação:** implementada com a *framework Javalin 6*, expõe uma *API REST* na porta 7000 e serve o *frontend* React através de *nginx* na porta 3000. Todos os *endpoints* estão agrupados em controllers por subsistema, registados em EcoRideAPI.
- **LN — Lógica de Negócio:** organizada em 8 subsistemas, cada um com a sua interface e respetivo facade (*SAutenticacaoFacade*, *SClientesFacade*, *SFuncionariosFacade*, *SStockFacade*, *SOrdensServicoFacade*, *SReparacoesFacade*, *SNotificacoesFacade*, *SFinanceiroFacade*). O ponto de entrada unificado é o *EcoRideController*, que implementa *IEcoRideController* e agrega todos facades dos subsistemas.
- **CD — Camada de Dados:** composta por *DAOs* (*Data Access Objects*) que implementam a interface padrão *Map<Integer, Entidade>* do Java. Cada *DAO* é um *singleton* que comunica com a base de dados **MySQL 8.4** via *JDBC*, através da classe *ConnectionFactory*.

A comunicação entre camadas é feita de forma unidirecional: a *CA* conhece a *LN* (através da interface *IEcoRideController*), e a *LN* conhece a *CD* (através das interfaces dos *DAOs*). A *CD* não conhece nenhuma das camadas superiores.

As tecnologias utilizadas no projeto são:

Componente	Tecnologia
Servidor <i>API</i>	Java 21 + Javalin 6
Base de dados	MySQL 8.4
<i>Frontend</i>	React + TypeScript + Vite + Tailwind CSS
Servidor <i>web</i>	nginx (alpine)
Containerização	Docker + Docker Compose
Testes unitários	JUnit 5 + Maven Surefire 3.2.5

Tabela 1: Tecnologias usadas no projeto



Figura 1: Uso dos subsistemas em EcoRideController

1.3. Pré-requisitos e Ambiente

Antes de colocar o sistema em funcionamento, é necessário garantir que o ambiente de execução cumpre os requisitos mínimos listados abaixo.

1.3.1. Requisitos para execução em Docker (recomendado)

- **Docker** versão 24 ou superior
- **Docker Compose** versão 2.20 ou superior

Não é necessário instalar Java, Node.js ou MySQL na máquina anfitriã pois todas as dependências são resolvidas dentro dos containers.

1.3.2. Requisitos para desenvolvimento local

- **Java 21** (*JDK*, para compilar e executar o backend diretamente)
- **Maven 3.9** (ou utilizar o *wrapper* incluído no projeto)
- **Node.js 22** e **npm** (para o *frontend*)
- **MySQL 8.4** acessível localmente, com uma base de dados chamada **EcoRide** criada

1.3.3. Variáveis de ambiente

O sistema é configurado inteiramente através de variáveis de ambiente, definidas no ficheiro `docker-compose.yml`. As variáveis obrigatórias para o serviço **backend** são:

Variável	Valor por omissão	Descrição
DB_HOST	mysql	Nome do host da base de dados
DB_PORT	3306	Porto <i>TCP</i> do <i>MySQL</i>
DB_NAME	EcoRide	Nome da base de dados
DB_USER	ecoride	Utilizador <i>MySQL</i>
DB_PASS	EcoRide123!	Password do utilizador <i>MySQL</i>
ENCRYPTION_KEY	—	Chave <i>AES-256-GCM</i> em Base64 (obrigatória)
DEV_MODE	false	Desliga autenticação se "true" (só desenvolvimento)

Atenção: a variável `ENCRYPTION_KEY` é crítica. A sua ausência impede o arranque correto do sistema, e a sua alteração após dados já inseridos torna os campos cifrados ilegíveis. Consultar a Secção 1.6 para mais detalhes.



```
1 package app.ecoRideCD.DAOconfig;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 import app.common.EcoRideException;
8
9 public final class ConnectionFactory {
10     private static final String HOST = env("DB_HOST", "localhost");
11     private static final String PORT = env("DB_PORT", "3306");
12     public static final String DATABASE = env("DB_NAME", "EcoRide");
13     public static final String USERNAME = env("DB_USER", "ecoride");
14     public static final String PASSWORD = env("DB_PASS", "EcoRide123!");
15     public static final String URL = "jdbc:mysql://" + HOST + ":" + PORT + "/" + DATABASE
16         + "?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true";
17
18     private static String env(String key, String fallback) {
19         String v = System.getenv(key);
20         return (v != null && !v.isBlank()) ? v : fallback;
21     }
22
23     private ConnectionFactory() {}
24
25     public static Connection get() {
26         try {
27             return DriverManager.getConnection(URL, USERNAME, PASSWORD);
28         } catch (SQLException e) {
29             throw new EcoRideException("Não foi possível obter ligação à BD", e);
30         }
31     }
32 }
33
```

Figura 2: Classe ConnectionFactory

1.4. Arranque e Paragem do Sistema

O sistema é inteiramente gerido pelo Docker Compose, que coordena 5 serviços declarados no ficheiro `docker-compose.yml`, na raiz do projeto.

1.4.1. Primeira execução

Para construir as imagens e iniciar todos os serviços pela primeira vez, terá de se executar o seguinte comando na diretoria `code/`:

```
docker compose up --build
```

Na primeira execução, o Docker irá:

1. Construir a imagem do backend, compilando usando o Maven em dois estágios: primeiro, descarrega dependências e compila o *JAR* e depois copia-o para uma imagem de runtime *JRE* 21;
2. Construir a imagem do frontend, instalando as dependências *npm*, e executa `npm run build` com Vite, e copia os ficheiros estáticos para uma imagem *nginx*;
3. Iniciar o serviço *mysql*, aguardar que o *healthcheck* confirme que está pronto a aceitar ligações;
4. Iniciar o *backend*, que ao arrancar executa `DBInitializer.run()` — aplica `schema.sql` (criação das tabelas) e `seed.sql` (dados iniciais), se existirem;
5. Iniciar o *frontend* e o *swagger* após o backend estar saudável;
6. Executar o serviço *info* que é um contentor de curta duração que imprime os *URLs* de acesso no terminal).

Após o arranque completo, os serviços ficam acessíveis nos seguintes endereços:

Serviço	URL
Interface <i>web</i>	http://localhost:3000
API direta	http://localhost:7000
Swagger <i>UI</i>	http://localhost:8081
Especificação <i>OpenAPI</i>	http://localhost:3000/api-docs

Tabela 2: Endereços de acesso aos serviços

1.4.2. Execuções subsequentes

Após a primeira build, basta executar:

```
docker compose up
```

Para reconstruir as imagens depois de alterações ao código:

```
docker compose up --build
```

1.4.3. Paragem

Para parar os serviços preservando os dados da base de dados:

```
docker compose down
```

Para parar e **eliminar completamente os dados** (volume `mysql_data`):

```
docker compose down -v
```

Atenção: o comando `down -v` é irreversível para os dados em `mysql_data`. Deverá ser usado apenas quando se pretende um ambiente completamente limpo ou se necessitar de recriar o esquema a partir do zero.

1.5. Base de Dados

A base de dados utilizada é *MySQL* 8.4, gerida pelo serviço `mysql` do Docker Compose. O esquema é declarado inteiramente no ficheiro `app/src/main/resources/schema.sql`, que é executado automaticamente no arranque da aplicação pela classe `DBInitializer`.

1.5.1. Inicialização automática

A classe `DBInitializer` é invocada na primeira linha do método `main` de `EcoRideAPI`. Ela lê `schema.sql` e `seed.sql` do *classpath*, remove comentários, divide as instruções pelo delimitador `;` e executa-as sequencialmente. Dado que todas as tabelas utilizam `CREATE TABLE IF NOT EXISTS`, o script é idempotente, já que pode ser executado múltiplas vezes sem erros nem perda de dados.

A screenshot of a code editor with a dark background and light-colored text. The code is in Java and defines a class named DBInitializer. The code is numbered from 1 to 28 on the left side. The class has a private constructor, a public static run() method, and a private static runScript() method. The run() method calls runScript() twice, once with "schema.sql" and once with "seed.sql". The runScript() method takes a String resource as input, reads it as a stream, removes comment lines, and then executes the SQL statements in a database connection.

```
1 public final class DBInitializer {
2
3     private DBInitializer() {}
4
5     public static void run() {
6         runScript("schema.sql");
7         runScript("seed.sql");
8     }
9
10    private static void runScript(String resource) {
11        try (InputStream in = DBInitializer.class.getClassLoader().getResourceAsStream(resource)) {
12            if (in == null) return;
13            String content = new String(in.readAllBytes(), StandardCharsets.UTF_8);
14
15            // Remove linhas de comentário antes de dividir por ";"
16            String cleaned = Arrays.stream(content.split("\n"))
17                .filter(line -> !line.strip().startsWith("--"))
18                .collect(Collectors.joining("\n"));
19
20            try (Connection c = ConnectionFactory.get();
21                Statement s = c.createStatement()) {
22                for (String stmt : cleaned.split(";")) {
23                    String trimmed = stmt.strip();
24                    if (!trimmed.isEmpty()) {
25                        s.execute(trimmed);
26                    }
27                }
28            }
29        }
30    }
31 }
```

Figura 3: Classe DBInitializer

1.5.2. Estrutura do esquema

O esquema organiza 26 tabelas pelos 8 subsistemas. Os padrões de modelação mais relevantes são:

- **Class Table Inheritance:** usado em `MovimentoFinanceiro` e `Notificacao`. Existe uma tabela base com os campos comuns e uma tabela filha por sub-tipo (`MovimentoFuncionario`, `MovimentoReparacao`, `MovimentoPeca`; `NotificacaoOS`, `NotificacaoStock`). A *fk* das tabelas filhas aponta para a base com `ON DELETE CASCADE`, pelo que apagar a linha base remove automaticamente a linha filha.
- **Tabelas de junção para coleções:** listas como `OrdemServico_Acessorio`, `Diagnostico_PecaOrcamento`, `Conserto_PecaUsada` e `Encomenda_EntradaStock` armazenam os elementos de listas Java (`List<String>`, `Map<Integer, Integer>`) com uma coluna `ordem` para preservar a posição.
- **Chaves primárias compostas:** em tabelas como `Diagnostico` e `Conserto`, o `idOS` serve simultaneamente como *PK* e *FK*, expressando a cardinalidade 0..1 com `OrdemServico`.

1.5.3. Acesso direto à base de dados

Para aceder ao *MySQL* em modo de manutenção enquanto o sistema está a correr, pode correr:

```
docker compose exec mysql mysql -uecoride -pEcoRide123! EcoRide
```


A partir daqui, estará na *bash* do *container* da base de dados e poderá executar comandos *sql* de acordo como bem necessitar.

1.6. Cifragem de Dados Sensíveis

Dado que a aplicação armazena dados pessoais de funcionários (nomes, documentos de identificação, dados financeiros e de morada), foi implementada uma camada de cifragem para os campos mais sensíveis da tabela `Funcionario`.

1.6.1. Campos cifrados

Os seguintes campos são cifrados antes de serem escritos na base de dados e decifrados aquando da leitura:

- nome
- telemovel
- email
- data_nascimento
- NISS
- NIF
- NUS
- IBAN
- salario_hora
- salario_liquido
- salario_bruto
- numero_porta
- rua
- localidade
- codigo_postal

Na tabela `Funcionario` do `schema.sql`, estes campos são declarados como `VARCHAR` com comprimento aumentado (100–400 caracteres) para acomodar o texto cifrado em *Base64*, que é consideravelmente mais longo que o valor original.

1.6.2. Algoritmo: *AES–256-GCM*

A cifragem é implementada pela classe utilitária `CifraUtil`, que utiliza o algoritmo *AES–256-GCM* (*Advanced Encryption Standard* com modo *Galois/Counter Mode*). Este modo garante simultaneamente **confidencialidade** e **integridade** dos dados, e, assim, qualquer adulteração do texto cifrado é detetada na decifragem.

Cada operação de cifragem gera um *IV* (*Initialization Vector*) aleatório de 12 bytes, que é concatenado com o texto cifrado antes da codificação em *Base64*. Isto garante que o mesmo valor em claro produz sempre um texto cifrado diferente.

```
1 public static String cifrar(String plaintext) {
2     if (plaintext == null) return null;
3     try {
4         byte[] iv = new byte[IV_LEN];
5         new SecureRandom().nextBytes(iv);
6
7         Cipher cipher = Cipher.getInstance(ALGORITHM);
8         cipher.init(Cipher.ENCRYPT_MODE, SECRET_KEY, new GCMParameterSpec(TAG_LEN, iv));
9         byte[] encrypted = cipher.doFinal(plaintext.getBytes("UTF-8"));
10
11         // IV + ciphertext+tag concatenados
12         byte[] out = new byte[IV_LEN + encrypted.length];
13         System.arraycopy(iv, 0, out, 0, IV_LEN);
14         System.arraycopy(encrypted, 0, out, IV_LEN, encrypted.length);
15         return Base64.getEncoder().encodeToString(out);
16     } catch (Exception e) {
17         throw new RuntimeException("Erro ao cifrar dado", e);
18     }
19 }
20
21 public static String decifrar(String ciphertext) {
22     if (ciphertext == null) return null;
23     try {
24         byte[] raw = Base64.getDecoder().decode(ciphertext);
25         byte[] iv = new byte[IV_LEN];
26         System.arraycopy(raw, 0, iv, 0, IV_LEN);
27         byte[] data = new byte[raw.length - IV_LEN];
28         System.arraycopy(raw, IV_LEN, data, 0, data.length);
29
30         Cipher cipher = Cipher.getInstance(ALGORITHM);
31         cipher.init(Cipher.DECRYPT_MODE, SECRET_KEY, new GCMParameterSpec(TAG_LEN, iv));
32         return new String(cipher.doFinal(data), "UTF-8");
33     } catch (Exception e) {
34         throw new RuntimeException("Erro ao decifrar dado", e);
35     }
36 }
```

```
1 public static String cifrar(String plaintext) {
2     if (plaintext == null) return null;
3     try {
4         byte[] iv = new byte[IV_LEN];
5         new SecureRandom().nextBytes(iv);
6
7         Cipher cipher = Cipher.getInstance(ALGORITHM);
8         cipher.init(Cipher.ENCRYPT_MODE, SECRET_KEY, new GCMParameterSpec(TAG_LEN, iv));
9         byte[] encrypted = cipher.doFinal(plaintext.getBytes("UTF-8"));
10
11         // IV + ciphertext+tag concatenados
12         byte[] out = new byte[IV_LEN + encrypted.length];
13         System.arraycopy(iv, 0, out, 0, IV_LEN);
14         System.arraycopy(encrypted, 0, out, IV_LEN, encrypted.length);
15         return Base64.getEncoder().encodeToString(out);
16     } catch (Exception e) {
17         throw new RuntimeException("Erro ao cifrar dado", e);
18     }
19 }
20
21 public static String decifrar(String ciphertext) {
22     if (ciphertext == null) return null;
23     try {
24         byte[] raw = Base64.getDecoder().decode(ciphertext);
25         byte[] iv = new byte[IV_LEN];
26         System.arraycopy(raw, 0, iv, 0, IV_LEN);
27         byte[] data = new byte[raw.length - IV_LEN];
28         System.arraycopy(raw, IV_LEN, data, 0, data.length);
29
30         Cipher cipher = Cipher.getInstance(ALGORITHM);
31         cipher.init(Cipher.DECRYPT_MODE, SECRET_KEY, new GCMParameterSpec(TAG_LEN, iv));
32         return new String(cipher.doFinal(data), "UTF-8");
33     } catch (Exception e) {
34         throw new RuntimeException("Erro ao decifrar dado", e);
35     }
36 }
```

Figura 4: Métodos de cifragem e decifragem

1.6.3. Gestão da chave de cifragem

A chave é fornecida ao sistema através da variável de ambiente `ENCRYPTION_KEY`, no formato *Base64* de 32 bytes (256 bits). Para gerar uma nova chave:

```
openssl rand -base64 32
```

É, no entanto, relevante mencionar que a chave de cifragem não pode ser alterada depois de existirem dados na base de dados, sem proceder a uma migração prévia que decifre todos os registos com a chave antiga e que os volte a cifrar com a nova. A alteração da chave sem migração tornaria todos os dados pessoais de funcionários permanentemente ilegíveis.

1.7. Segurança e Autenticação

A segurança do sistema assenta em dois pilares: o controlo de acesso por token de sessão e a autorização por cargo. Ambos são implementados na camada *CA*, antes de qualquer lógica de negócio ser executada.

1.7.1. Middleware de autenticação

Em EcoRideAPI, é registrado um filtro global `app.before("/api/*", ...)` que interceta todos os pedidos para rotas protegidas. Em **modo de produção** (`DEV_MODE=false`), o filtro:

1. Lê o cabeçalho `Authorization` do pedido;
2. Valida o token no `GestorSesoes`;
3. Se válido, injeta o objeto `SessaoUtilizador` no contexto do pedido (`ctx.attribute("sessao", sessao)`);
4. Se inválido ou ausente, devolve HTTP 401 Unauthorized.

Em modo de desenvolvimento (`DEV_MODE=true`), o filtro injeta automaticamente uma sessão de `Gerente` em todos os pedidos, dispensando a autenticação. Este modo nunca deve ser ativado em produção.

```
1 boolean devMode = "true".equalsIgnoreCase(System.getenv("DEV_MODE"));
2 if (devMode) {
3     System.out.println("[EcoRide] DEV_MODE activo – autenticação desligada");
4     app.before("/api/*", ctx -> {
5         // Injeta sessão de Gerente para que verifica_cargo não lance 403
6         ctx.attribute("sessao", SessaoUtilizador.devSession());
7     });
8 } else {
9     app.before("/api/*", ctx -> {
10        String token = ctx.header("Authorization");
11        if (token == null) throw new UnauthorizedResponse("Token em falta");
12        SessaoUtilizador sessao = gestorSesoes.validar(token);
13        ctx.attribute("sessao", sessao);
14    });
15 }
```



Figura 5: Middleware de autenticação

1.7.2. Cargos e permissões

Existem 4 cargos no sistema, definidos no enum Cargo:

Cargo	Responsabilidades principais
Gerente	Acesso total; gestão de funcionários, utilizadores e financeiro
GestorStock	Gestão de stock, peças, fornecedores e encomendas
Secretaria	Registo de clientes, trotinetes e ordens de serviço
Mecanico	Diagnóstico, conserto e transições de estado das OS

Tabela 3: Cargos e responsabilidades

Cada controller verifica o cargo da sessão injetada antes de executar operações sensíveis, devolvendo HTTP 403 Forbidden em caso de acesso não autorizado.

1.7.3. Sessões

O GestorSesoes mantém um mapa em memória de tokens ativos. Os tokens são gerados no *endpoint* de autenticação (POST /auth/login) e invalidados em (POST /auth/logout).

Nota de manutenção: dado que as sessões são armazenadas em memória, o reinício do serviço backend invalida todas as sessões ativas, assim, os utilizadores terão de autenticar-se novamente.

1.8. API REST

O backend expõe uma *API REST* completa na porta 7000, documentada em formato **OpenAPI 3** no ficheiro `EcoRideCA/openapi.yaml`. A especificação interativa está disponível em <http://localhost:8081> (Swagger UI) e a especificação bruta em <http://localhost:3000/api-docs>.

1.8.1. Estrutura de rotas

As rotas seguem a convenção `/<prefixo>/<recurso>`, agrupadas por subsistema:

Prefixo	Subsistema
<code>/auth/</code>	Autenticação (login/logout)
<code>/api/clientes</code>	Clientes e trotinetes
<code>/api/funcionarios</code>	Funcionários
<code>/api/users</code>	Utilizadores do sistema
<code>/api/stock</code>	Pecas, stock, fornecedores, encomendas, defeitos
<code>/api/ordens-servico</code>	Ordens de serviço
<code>/api/reparacoes</code>	Catálogo de reparações
<code>/api/notificacoes</code>	Notificações
<code>/api/financeiro</code>	Movimentos financeiros e análise

Tabela 4: Rotas disponibilizadas para interações com os subsistemas

1.8.2. Tratamento de erros

Os erros de negócio são encapsulados na classe `EcoRideException` (exceção não verificada). O `GlobalExceptionHandler`, registado em `EcoRideAPI`, interceta estas exceções e devolve respostas *JSON* estruturadas com o código *HTTP* adequado (400 Bad Request para erros de validação, 404 Not Found para recursos inexistentes, 409 Conflict para conflitos de dados).

1.8.3. Adicionar um novo endpoint

Para expor uma nova operação na *API*, dever-se-á:

1. Adicionar o método à interface `IEcoRideController` e implementá-lo em `EcoRideController`;
2. Delegar na fachada do subsistema correspondente;
3. Criar ou editar o controller em `app/IEcoRideLN/controllers/<subsistema>/`;
4. Registrar o controller em `EcoRideAPI.main()`;
5. Documentar o novo endpoint em `openapi.yaml`.

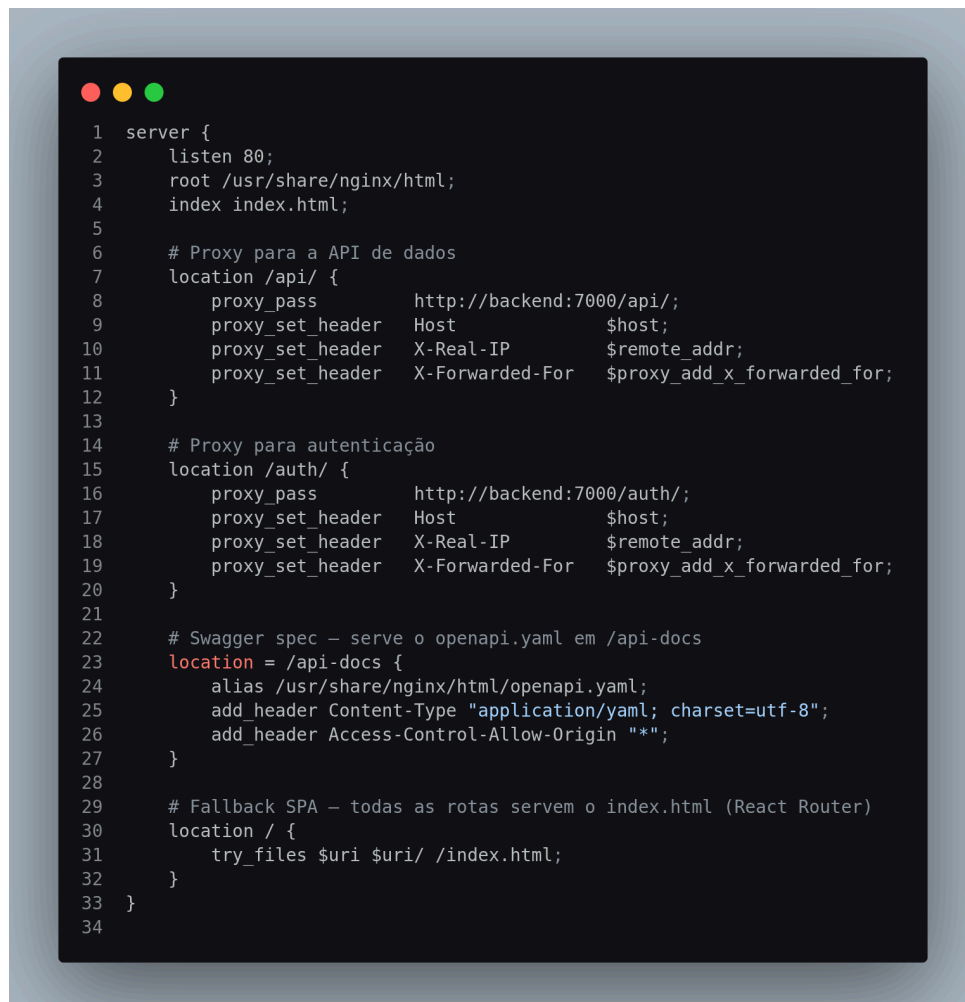
1.9. 8. *Frontend*

O *frontend* é uma aplicação de página única (*SPA*) desenvolvida em React com TypeScript, construída com Vite e estilizada com Tailwind CSS. Em produção, é servida pelo nginx, que atua simultaneamente como servidor de ficheiros estáticos e como proxy reverso para o backend.

1.9.1. Configuração do nginx

O ficheiro `EcoRideCA/nginx.conf` define o comportamento do nginx:

- Rotas `/api/*` e `/auth/*` são reencaminhadas para `http://backend:7000`;
- A directiva `resolver 127.0.0.11 valid=30s ipv6=off` instrui o nginx a usar o *DNS* interno do Docker, resolvendo o nome `backend` por pedido em vez de no arranque (evita falhas se o `backend` não estiver ainda disponível);
- Todas as restantes rotas devolvem `index.html` (necessário para o React Router funcionar em navegação direta).



```
1 server {
2     listen 80;
3     root /usr/share/nginx/html;
4     index index.html;
5
6     # Proxy para a API de dados
7     location /api/ {
8         proxy_pass          http://backend:7000/api/;
9         proxy_set_header    Host      $host;
10        proxy_set_header     X-Real-IP $remote_addr;
11        proxy_set_header     X-Forwarded-For $proxy_add_x_forwarded_for;
12    }
13
14    # Proxy para autenticação
15    location /auth/ {
16        proxy_pass          http://backend:7000/auth/;
17        proxy_set_header    Host      $host;
18        proxy_set_header     X-Real-IP $remote_addr;
19        proxy_set_header     X-Forwarded-For $proxy_add_x_forwarded_for;
20    }
21
22    # Swagger spec – serve o openapi.yaml em /api-docs
23    location = /api-docs {
24        alias /usr/share/nginx/html/openapi.yaml;
25        add_header Content-Type "application/yaml; charset=utf-8";
26        add_header Access-Control-Allow-Origin "*";
27    }
28
29    # Fallback SPA – todas as rotas servem o index.html (React Router)
30    location / {
31        try_files $uri $uri/ /index.html;
32    }
33 }
34
```

Figura 6: Configuração do nginx

1.9.2. Build local do *frontend*

Para compilar o *frontend* fora do Docker poderá:

```
cd code/EcoRideCA
npm ci
npm run build      # produz a pasta dist/
npm run dev        # servidor de desenvolvimento na porta 5173
```

1.9.3. Processo de *build* em Docker

O *Dockerfile* do *frontend* usa um *build* em dois estágios: o primeiro estágio usa a imagem `node:22-alpine` para compilar a aplicação; o segundo copia apenas os ficheiros estáticos resultantes para uma imagem `nginx:alpine`, sem incluir o Node.js nem as dependências de desenvolvimento na imagem final.

1.10. Execução de Testes

A estratégia de testes do sistema baseia-se em testes unitários automatizados, executados com a *framework* *JUnit* 5 através do *plugin* Maven `maven-surefire` 3.2.5. Os testes não requerem base de dados nem Docker — cada *DAO* real é substituído por uma subclasse anónima em memória (ver Secção 1.10.2). Assim, poderam-se obter testes rápidos, simples e eficazes sem mudar a lógica de negócio ou a estrutura interna do *facade* responsável por ela.

1.10.1. Executar os testes

Para executar a totalidade dos testes:

```
cd code/app
mvn test
```

O resultado é apresentado por classe de teste, com um sumário final dos testes corridos e aqueles que: sucederam, falharam, lançaram erros ou foram ignorados.

O ponto de entrada unificado é a classe `EcoRideTestSuite`, anotada com `@Suite` e `@SelectClasses`, que agrega todas as classes de teste.

1.10.2. Padrão de *DAO* falso em memória

Dado que todos os *DAOs* implementam `Map<Integer, Entidade>`, é possível criar subclasses anónimas que sobrepõem todos os métodos com implementações baseadas em `HashMap`, sem qualquer ligação à base de dados. O auto-incremento de *IDs* é simulado com `AtomicInteger`.



```
1 static FuncionarioDAO daoFake() {
2     return new FuncionarioDAO() {
3         private final Map<Integer, Funcionario> store = new HashMap<>();
4         private final AtomicInteger seq = new AtomicInteger(1);
5
6         @Override public int size() { return store.size(); }
7         @Override public boolean isEmpty() { return store.isEmpty(); }
8         @Override public boolean containsKey(Object k) { return store.containsKey(k); }
9         @Override public boolean containsValue(Object v) { return store.containsValue(v); }
10        @Override public Funcionario get(Object k) { return store.get(k); }
11        @Override public Funcionario put(Integer k, Funcionario v) { return store.put(k, v); }
12        @Override public Funcionario remove(Object k) { return store.remove(k); }
13        @Override public void putAll(Map<? extends Integer, ? extends Funcionario> m) { store.putAll(m); }
14        @Override public void clear() { store.clear(); }
15        @Override public Set<Integer> keySet() { return store.keySet(); }
16        @Override public Collection<Funcionario> values() { return store.values(); }
17        @Override public Set<Map.Entry<Integer, Funcionario>> entrySet() { return store.entrySet(); }
18
19        @Override
20        public int insert(Funcionario f) {
21            int id = seq.getAndIncrement();
22            f.setId(id);
23            store.put(id, f);
24            return id;
25        }
26    };
27 }
```

Figura 7: Exemplo de DAO usado em testes

1.10.3. Adicionar testes a um subsistema existente

Para adicionar novos casos de teste deverá:

1. Abrir a classe de teste correspondente ao subsistema (ex: `Teste11.java`);
2. Adicionar um método anotado com `@Test`, `@Order(n)` e `@DisplayName("...")`;
3. Usar `assertThrows` para casos de exceção esperada e `assertEquals/assertTrue` para verificações de estado;
4. Os novos testes serão automaticamente incluídos na próxima execução de `mvn test`, não sendo necessário alterar a `EcoRideTestSuite`.

1.11. Adição de um Novo Subsistema

Quando for necessário adicionar um subsistema completamente novo ao **EcoRide**, deverão ser seguidos os seguintes passos, pela ordem indicada:

1. **Entidade LN**: criar a classe da entidade em `app/ecoRideLN/s<Nome>/`, com construtores, *getters* e *setters*.
2. **DAO**: criar a classe `<Nome>DAO` em `app/ecoRideCD/s<Nome>/`, implementando `Map<Integer, <Nome>>`. O construtor deverá ser `protected` (para permitir subclasses em testes) e deverá implementar os métodos de domínio específicos (`insert`, métodos de pesquisa, etc.).
3. **Interface e Fachada LN**: criar `IS<Nome>` com os métodos de negócio e `S<Nome>Facade` que os implementa. Adicionar um construtor de injeção `public S<Nome>Facade(<Nome>DAO dao)` para suporte a testes.

4. **Integrar no `EcoRideController`:** adicionar o campo `IS<Nome>`, instanciá-lo no construtor e delegar os novos métodos na interface `IEcoRideController`.
5. **Controller `CA`:** criar `<Nome>Controller` em `app/IEcoRideLN/controllers/<nome>/`, com os *endpoints REST* pretendidos. Registrar com `new <Nome>Controller(facade).register(app)` em `EcoRideAPI`.
6. **Esquema `SQL`:** adicionar as novas tabelas a `schema.sql`, no final do ficheiro, com `CREATE TABLE IF NOT EXISTS`. Respeitar as dependências de chaves estrangeiras.
7. **Testes:** criar `Teste<Nome>.java` em `app/src/test/java/app/ecoRideLN/`, seguindo o padrão dos ficheiros existentes. Adicionar a classe à anotação `@SelectClasses` de `EcoRideTestSuite`.
8. **Documentação `API`:** adicionar os novos *endpoints* ao ficheiro `openapi.yaml`.

1.12. Monitorização e Logs

Em ambiente Docker, todos os logs são acessíveis através do Docker Compose.

1.12.1. Visualizar logs em tempo real

Para seguir os logs de todos os serviços em simultâneo pode executar:

```
docker compose logs -f
```

Para um serviço específico:

```
docker compose logs -f backend
docker compose logs -f mysql
docker compose logs -f frontend
```

1.12.2. Mensagens de arranque relevantes

O serviço `backend` emite as seguintes mensagens durante o arranque, úteis para diagnóstico:

- `[DBInitializer]` Aviso ao executar `schema.sql`: ... — indica que o script *SQL* falhou numa instrução. Normalmente inofensivo se o esquema já existia.
- `[EcoRide]` `DEV_MODE` activo – autenticação desligada — confirma que o modo de desenvolvimento está ativo. Não deve aparecer em produção.
- Ausência de mensagens de erro do Javalin na porta 7000 indica arranque bem sucedido.

1.12.3. Estado dos containers

Para verificar o estado e saúde de todos os containers pode correr:

```
docker compose ps
```

O campo `STATUS` deverá indicar `running (healthy)` para os serviços `mysql` e `backend`. O serviço `info` deverá aparecer como `exited (0)` após imprimir os *URLs* de acesso — este comportamento é esperado.